Sketch-to-Image Generator

Deep Learning Concepts & Algorithms Used

Conditional GAN (Pix2Pix) · U-Net Generator · PatchGAN Discriminator

Deep Learning & Generative AI, CA-2 Project
Track A: Generative Adversarial Networks (Unit 3)

Team member	Roll number	Section

Contents

#	Section
1	Project overview and end-to-end pipeline
2	Neural-network foundations: neurons, backpropagation, gradient descent
3	Convolutional building blocks: convolution, transposed convolution, activations, normalisation, dropout, init
4	Generative Adversarial Networks: minimax game, training algorithm
5	Conditional GANs and the Pix2Pix objective (cGAN + L1)
6	Generator architecture: encoder-decoder and U-Net skip connections
7	Discriminator architecture: PatchGAN and receptive fields
8	Dataset, sketch extraction (XDoG, Canny) and preprocessing
9	Optimisation: Adam, LR schedule, alternating updates, mixed precision
10	GAN training stability: failure modes and fixes
11	Evaluation metrics: L1, PSNR, SSIM, FID; ablation study
12	Inference and deployment
13	Hyper-parameter summary and concept-to-code map
14	Viva preparation: likely questions and answers
15	References

This document explains every deep-learning idea used in the project repository. For each concept it gives the intuition, the maths, and where it appears in the code (green boxes). Track A covers image-to-image translation with Pix2Pix, conditional image generation (cGAN), and DCGAN design principles.

1. Project overview

Problem statement

Designers, artists and e-commerce teams often begin with a rough line drawing. Turning that sketch into a realistic, coloured image takes skill and time. The task is **image-to-image translation**: learn a mapping G: sketch → photo from paired examples, so that a new hand-drawn outline produces a plausible image with correct shape, texture, colour and shading.

This is ill-posed, because one sketch has many valid colourings. A plain regression network trained with pixel loss averages over all of them and gives blurry, grey output. A **conditional GAN** fixes this: a learned critic (the discriminator) penalises anything that does not look real, which pushes the generator to commit to one sharp, realistic answer.

Motivation

- Rapid prototyping for product design (shoes, bags, apparel) and concept art.
- Data augmentation: generate many photo-realistic variants from cheap line art.
- A clear demonstration of the core Unit 3 ideas: adversarial training, conditioning, U-Net, PatchGAN.

End-to-end pipeline

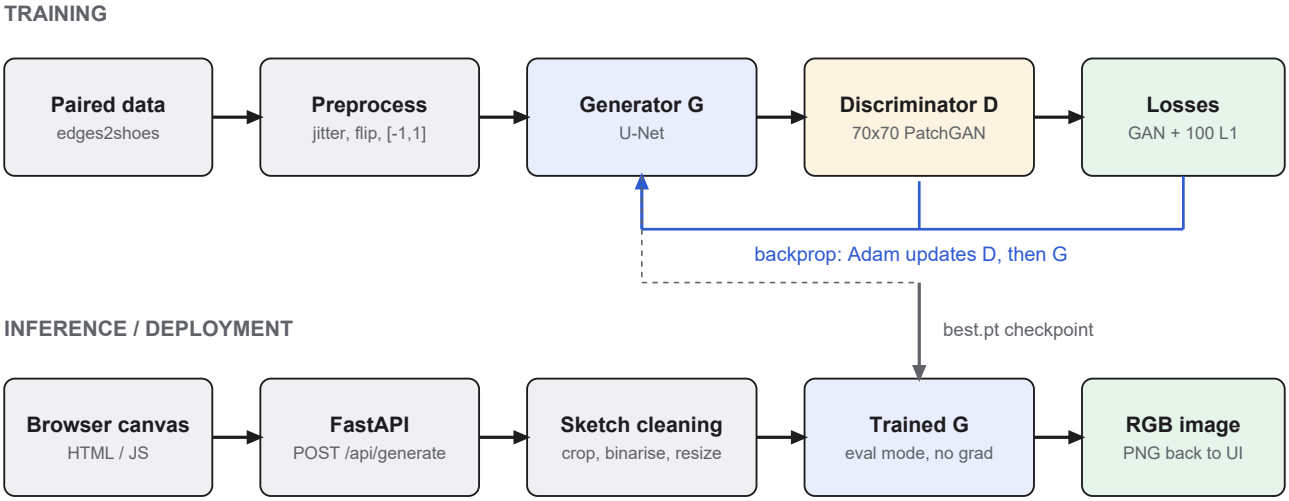


Figure 1. Training (top) learns G and D jointly. Deployment (bottom) uses only the trained generator behind a FastAPI endpoint called from a browser drawing canvas.

2. Neural-network foundations

2.1 Artificial neuron and deep networks

A neuron computes a weighted sum of its inputs plus a bias, then applies a non-linear activation. Stacking layers of neurons gives a deep network f(x; θ) that can approximate very complex functions (universal approximation). Without the non-linearity, any stack of layers would collapse into one linear map.

$$z = W \cdot x + b, \qquad a = \varphi(z)$$

2.2 Loss function and empirical risk

Training minimises the average loss over the dataset, which measures how far predictions are from the targets. This project uses three kinds of loss: **binary cross-entropy** (adversarial), **L1 / mean absolute error** (reconstruction) and, optionally, a **perceptual** feature loss.

$$\theta^* = \arg\min_{\theta} \; (1/N) \, \Sigma_i \, L(\, f(x_i; \, \theta), \, y_i \,)$$

2.3 Backpropagation

Backpropagation applies the chain rule from the loss back to every weight, giving $\partial L/\partial \theta$ for every parameter in one backward pass. PyTorch's **autograd** records the forward computation graph and calls `loss.backward()` to fill each parameter's `.grad`.

$$\partial L/\partial W^{(l)} = \delta^{(l)} \cdot a^{(l-1)T}, \qquad \delta^{(l)} = (W^{(l+1)T} \, \delta^{(l+1)}) \odot \varphi'(z^{(l)})$$

2.4 Gradient descent and mini-batches

Parameters move a small step against the gradient. **Mini-batch stochastic gradient descent** estimates the gradient from a small batch (8 images here), which is cheap and adds helpful noise. An **epoch** is one full pass over the training set.

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \, L_{batch}(\theta)$$

In our code: `DataLoader(batch_size=8, shuffle=True)` in `train.py`; the update rule is Adam (Section 9), a refined form of gradient descent.

3. Convolutional building blocks

3.1 2-D convolution

A convolution slides a small learnable kernel over the image. Each output value is a dot product between the kernel and a local patch. Two properties make CNNs the standard model for images: **weight sharing** (the same kernel is used at every position, so there are far fewer parameters) and **local connectivity** (nearby pixels are processed together, which builds in translation equivariance).

$$Y[i, j] = \Sigma_c \Sigma_{u,v} K[c, u, v] \cdot X[c, s{\cdot}i + u - p, s{\cdot}j + v - p] + b$$
$$H_{out} = \text{floor}((H_{in} + 2p - k) / s) + 1$$

With kernel k=4, stride s=2 and padding p=1, every encoder layer exactly halves the resolution (256→128→…→1). **Strided convolution** replaces max-pooling, so the network learns its own down-sampling. This is one of the DCGAN guidelines.

3.2 Transposed convolution (up-sampling)

Transposed convolution, sometimes called deconvolution, is the gradient of a convolution with respect to its input. Each input pixel scatters a weighted copy of the kernel into a larger output, so it **learns** how to up-sample. With k=4, s=2, p=1 it exactly doubles H and W.

$$H_{out} = (H_{in} - 1){\cdot}s - 2p + k \;=\; (H_{in} - 1){\cdot}2 - 2 + 4 \;=\; 2{\cdot}H_{in}$$

3.3 Activation functions

Function	Formula	Where used and why
ReLU	$\max(0, z)$	Decoder blocks of G. Cheap and does not saturate for $z > 0$.
LeakyReLU (α = 0.2)	$\max(\alpha z, z)$	Encoder of G and all layers of D. Keeps a small gradient for $z < 0$, avoids 'dead' units and keeps gradients flowing to G (DCGAN guideline).
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	Output layer of G. Bounded to $[-1, 1]$, matching how the images are normalised.
Sigmoid	$1 / (1 + e^{-z})$	Folded into BCEWithLogitsLoss, which turns D's logits into probabilities. Using the fused form is numerically stable.

3.4 Batch Normalisation and Instance Normalisation

BatchNorm normalises each channel to zero mean and unit variance using batch statistics, then applies a learnable scale γ and shift β. It smooths the loss landscape, allows higher learning rates and stabilises GAN training. **InstanceNorm** normalises each image separately, which is common in style and translation models and is available as `--norm instance`.

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)}, \qquad y = \gamma{\cdot}\hat{x} + \beta$$

Following Pix2Pix, there is **no normalisation** on the first encoder layer, on the 1×1 bottleneck (a single value per channel cannot be normalised), or on the first layer of D.

3.5 Dropout as the noise source

Dropout randomly zeroes activations with probability p = 0.5. Normally it is a regulariser. In Pix2Pix it also acts as the **noise z** of the GAN: a random noise vector fed to G is simply ignored, so the randomness is injected through dropout in the first three decoder blocks.

3.6 Weight initialisation

Every conv and transposed-conv weight is drawn from N(0, 0.02) and every norm scale γ from N(1, 0.02) (DCGAN / Pix2Pix). Small, symmetric initial weights keep early activations and gradients in a healthy range, so neither network dominates at the start.

In our code: `Down`, `Up`, `init_weights()` in `s2i/models.py`.

4. Generative Adversarial Networks

4.1 Generative vs discriminative models

A discriminative model learns p(label | data). A **generative** model learns to produce samples from the data distribution p_{data}. GANs (Goodfellow et al., 2014) do this *implicitly*: there is no explicit likelihood. Instead, a generator G is trained to fool a discriminator D, and D is trained to tell real samples from generated ones.

4.2 The minimax game

$$\min_G \max_D \; V(D, G) = E_{y \sim p_{data}}[\log D(y)] + E_{z \sim p_z}[\log(1 - D(G(z)))]$$

For a fixed G, the optimal discriminator is $D^*(y) = p_{data}(y) / (p_{data}(y) + p_g(y))$. Substituting D^* back in shows that G minimises the **Jensen–Shannon divergence** between the real and generated distributions. The global optimum is $p_g = p_{data}$, where D outputs 0.5 everywhere (a Nash equilibrium).

$$V(D^*, G) = -\log 4 + 2 \cdot \mathrm{JSD}(p_{data} \,\|\, p_g)$$

4.3 Non-saturating generator loss

Early in training D easily rejects fakes, so log(1 − D(G(z))) saturates and gives almost no gradient. In practice G instead **maximises log D(G(z))**, which is equivalent to BCE with the target 'real'. This has the same fixed point but much stronger gradients. Our code does exactly this: `gan_loss(D(sketch, fake), True)`.

4.4 GAN training algorithm (alternating SGD)

```
for each mini-batch (x, y):
    # --- Discriminator step: ascend log D(x,y) + log(1 - D(x, G(x))) ---
    y_fake = G(x)
    L_D = 0.5 * [ BCE(D(x, y), 1) + BCE(D(x, y_fake.detach()), 0) ]
    update D with Adam on grad(L_D)

    # --- Generator step: fool D and stay close to the ground truth ---
    freeze D
    L_G = BCE(D(x, y_fake), 1) + lambda_L1 * |y - y_fake|_1
    update G with Adam on grad(L_G)
    unfreeze D
```

`detach()` stops gradients from L_D flowing into G. Freezing D (`requires_grad=False`) during the G step saves memory and compute, since only G's weights change there. The factor 0.5 on L_D slows D down relative to G, as in Pix2Pix.

4.5 DCGAN design principles used

- Strided convolutions instead of pooling, and transposed convolutions for up-sampling.
- Batch normalisation in both G and D (except the input/output layers).
- ReLU in G (Tanh at the output), LeakyReLU in D.
- Adam with lr = $2 \cdot 10^{-4}$ and $\beta_1 = 0.5$; weights initialised from N(0, 0.02).

5. Conditional GAN and the Pix2Pix objective

5.1 Conditional GAN (cGAN)

A cGAN (Mirza & Osindero, 2014) feeds extra information x to **both** networks. Here x is the input sketch. G learns p(y | x) instead of p(y), and D judges whether the pair (x, y) is a real match, not just whether y looks real. That is what forces the output to **follow the sketch's lines**.

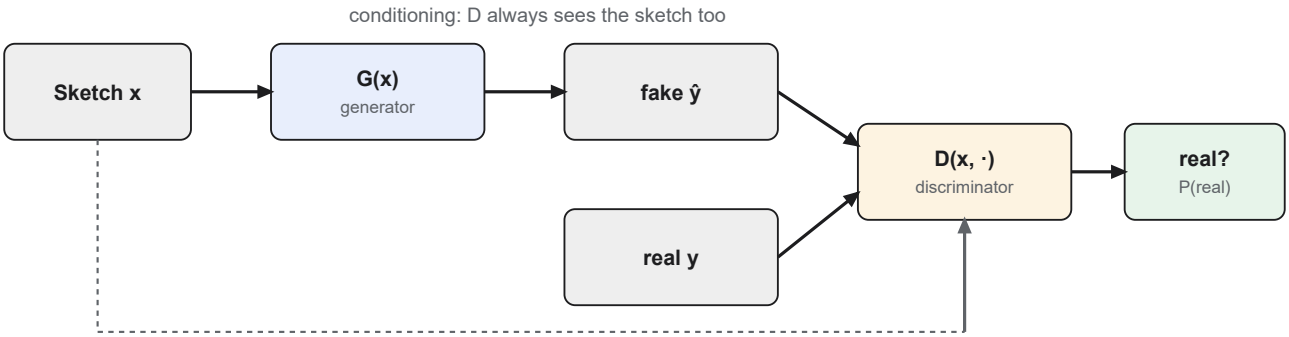


Figure 2. In a conditional GAN, D receives the sketch concatenated with either the real photo or G's output (4 channels in total).

$$L_{cGAN}(G, D) = E_{x,y}[\log D(x, y)] + E_x[\log(1 - D(x, G(x)))]$$

5.2 Adding a reconstruction loss (L1)

Because training data is paired, G also gets a direct supervised signal: the L1 distance to the ground-truth photo. L1 captures **low-frequency** correctness (overall colour and layout). The adversarial loss supplies **high-frequency** realism (sharp edges, texture). L1 is preferred over L2 because it is less blurry: its minimiser is the median rather than the mean.

$$L_{L1}(G) = E_{x,y}[\, \| y - G(x) \|_1 \,]$$

5.3 Final objective

$$G^* = \arg\min_G \max_D \; L_{cGAN}(G, D) + \lambda \cdot L_{L1}(G), \qquad \lambda = 100$$

$\lambda = 100$ balances the two terms. The L1 loss on [−1, 1] images is around 0.1–0.3 while the GAN loss is around 1, so scaling L1 by 100 makes the reconstruction signal dominant early on and keeps training stable. The adversarial term then sharpens the result.

5.4 Optional extensions implemented

Flag	Concept	Effect
--gan-mode lsgan	Least-Squares GAN (Mao et al., 2017): L = (D − target)^2	Penalises samples by distance from the decision boundary. Smoother gradients, less vanishing.
--lambda-vgg 10	Perceptual loss: L1 between VGG-16 feature maps of fake and real	Transfer learning from ImageNet. Matches textures and structures rather than raw pixels.
--real-label 0.9	One-sided label smoothing (Salimans et al., 2016)	Stops D from becoming over-confident, which keeps G's gradients informative.
--lambda-gan 0	Ablation: pure L1 regression (no GAN)	Baseline showing what the adversarial loss adds (Section 11).

In our code: `GANLoss` and `VGGPerceptualLoss` in `s2i/losses.py`; the combined objective is built in the G step of `train.py`.

6. Generator: U-Net encoder-decoder

6.1 Encoder-decoder

The encoder repeatedly halves the spatial resolution while increasing the channels, compressing the sketch into a 1×1×512 **bottleneck** that encodes global context (what object it is, what style). The decoder mirrors this to reconstruct a full-resolution image.

6.2 Why skip connections (U-Net)?

In sketch-to-image translation, the input and output share the same **structure**: edges in the sketch must stay in the same place in the photo. Squeezing everything through a 1×1 bottleneck would lose those exact positions. A **U-Net** (Ronneberger et al., 2015) concatenates each encoder layer i with the decoder layer n−i, so fine spatial detail skips the bottleneck. Skip connections also shorten the gradient path, which makes a 16-layer network easy to train.

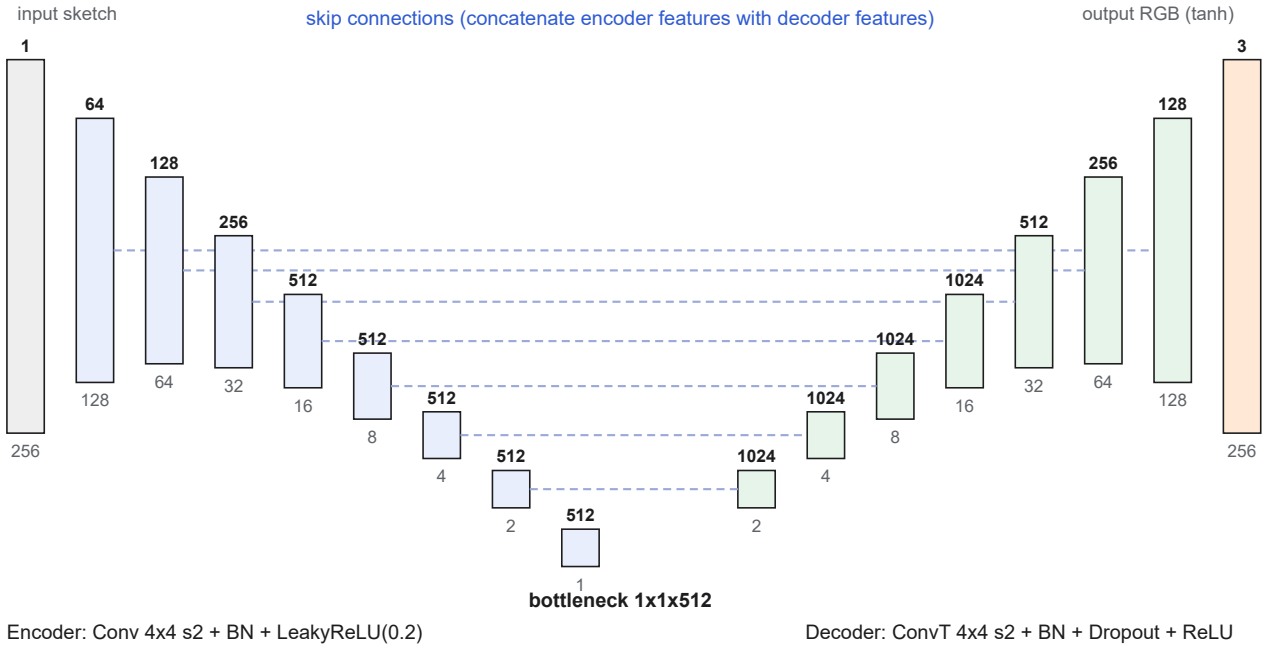


Figure 3. U-Net generator for a 256×256 input. Numbers above the blocks are channels; numbers below are spatial size. Decoder blocks show channels after concatenation.

6.3 Layer-by-layer shapes

Block	Operation	Output shape	Notes
input	sketch	1 × 256 × 256	grayscale, in [−1, 1]
D1	Conv4 s2, LReLU	64 × 128 × 128	no norm
D2–D4	Conv4 s2, BN, LReLU	128×64², 256×32², 512×16²	
D5–D7	Conv4 s2, BN, LReLU	512 × 8², 4², 2²	
D8	Conv4 s2, LReLU	512 × 1 × 1	bottleneck, no norm
U1–U3	ConvT4 s2, BN, Dropout 0.5, ReLU, concat	1024 × 2², 4², 8²	noise via dropout
U4	ConvT4 s2, BN, ReLU, concat	1024 × 16²	
U5–U7	ConvT4 s2, BN, ReLU, concat	512×32², 256×64², 128×128²	
out	ConvT4 s2, Tanh	3 × 256 × 256	RGB in [−1, 1]

Trainable parameters: **54.41 M**. The number of down-sampling steps is computed as $\log_2$(image size), so the same code works at 64, 128 or 256 px.

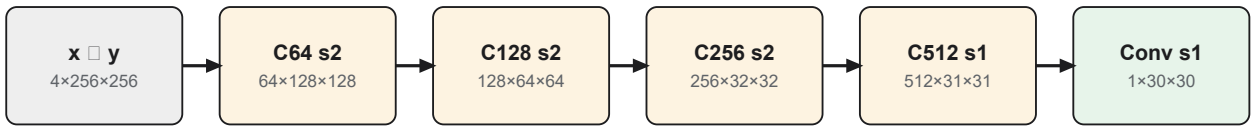
In our code: `UNetGenerator.forward()` pushes each encoder output onto a stack and pops them in reverse order for concatenation (`torch.cat([x, skips.pop()], 1)`).

7. Discriminator: PatchGAN

7.1 Idea

Instead of one real/fake score for the whole image, a **PatchGAN** is a fully convolutional classifier that outputs an N×N grid. Each cell judges one local patch. The final score is the average loss over all patches. Because L1 already enforces low-frequency correctness, D only needs to model **high-frequency local texture**. In effect it treats the image as a Markov random field, assuming pixels more than a patch apart are independent.

- Far fewer parameters (2.77 M) than a full-image classifier, so it is fast and less prone to over-fitting.
- Works on any image size, because it is fully convolutional.
- Produces sharp detail everywhere in the image instead of a single global judgement.



each of the 30×30 outputs = real/fake logit for one 70×70 patch of the input

Figure 4. The 70×70 PatchGAN. Input is the sketch concatenated with a real or fake image.

7.2 Receptive-field calculation (why '70×70')

The receptive field r of an output unit is computed backwards from the last layer using $r_{in} = (r_{out} - 1)\cdot s + k$:

Layer (from output)	kernel k	stride s	receptive field r
output unit	–	–	1
Conv → 1 ch	4	1	(1−1)·1 + 4 = 4

Layer (from output)	kernel k	stride s	receptive field r
C512	4	1	(4−1)·1 + 4 = 7
C256	4	2	(7−1)·2 + 4 = 16
C128	4	2	(16−1)·2 + 4 = 34
C64	4	2	(34−1)·2 + 4 = **70**

In our code: `PatchDiscriminator(in_ch=4, ndf=64, n_layers=3)` outputs raw logits of shape (B, 1, 30, 30); `GANLoss` compares them with an all-ones or all-zeros tensor of the same shape.

8. Dataset and preprocessing

8.1 Dataset: edges2shoes

The main dataset is **edges2shoes** from the Pix2Pix authors: 50,025 training and 200 validation pairs built from UT-Zappos50K shoe photos. The sketches are edge maps made with the HED (Holistically-nested Edge Detection) CNN and post-processed into clean lines. Each file is 512×256, with the sketch on the left half and the photo on the right. edges2handbags (137k pairs) uses the same format. A random subset (`--max-train`, e.g. 10k) keeps training feasible on a 6 GB laptop GPU.

8.2 Custom domains: XDoG sketch synthesis

To train on any domain, `scripts/prepare_dataset.py` builds pairs automatically from photos with the **eXtended Difference-of-Gaussians (XDoG)** operator. A difference of two Gaussian blurs is a band-pass filter that approximates the Laplacian of Gaussian, so it responds to edges. A soft tanh threshold then turns the response into pencil-like strokes.

$$D(x) = G_{\sigma}(x) - \gamma \cdot G_{k\sigma}(x)$$
$$T(x) = 1 \ \text{ if } D(x) \geq \varepsilon, \quad \text{else } \ 1 + \tanh(\ \varphi \cdot (D(x) - \varepsilon)\ )$$
$$\sigma = 0.8,\ \ k = 1.6,\ \ \gamma = 0.98,\ \ \varepsilon = -0.1,\ \ \varphi = 200$$

Canny is offered as an alternative. It applies Gaussian smoothing, computes Sobel gradient magnitude and direction, thins edges with non-maximum suppression, and links them with double-threshold hysteresis (thresholds 60 / 150).

8.3 Preprocessing and augmentation

Step	Detail	Reason
Split pair	left half → sketch (1 ch), right half → photo (3 ch)	aligned A|B format
Normalise	$x' = 2 \cdot x/255 - 1 \rightarrow [-1, 1]$	matches Tanh output range and centres the data
Random jitter	resize to 286×286, random crop 256×256	translation robustness; Pix2Pix recipe
Horizontal flip	p = 0.5	doubles the effective data, since shoes are mirror-symmetric
Paired transform	the SAME crop/flip is applied to sketch and photo	keeps pixel alignment for L1
Validation	plain resize to 256, no augmentation	deterministic metrics

8.4 Cleaning hand-drawn sketches (inference)

User drawings differ from training edge maps (line thickness, position, scale). This **domain gap** is reduced by: flattening transparency onto white, auto-inverting light-on-dark input, cropping to the stroke bounding box, centring on a square canvas with a 15% margin, area resampling to 256 px, and binarising at threshold 180.

In our code: `PairedSketchDataset` in `s2i/dataset.py`; `xdog()`, `canny()` and `clean_user_sketch()` in `s2i/sketch.py`.

9. Optimisation and training details

9.1 Adam optimiser

Adam (Kingma & Ba, 2015) keeps exponential moving averages of the gradient (first moment m) and of the squared gradient (second moment v). This gives each parameter its own adaptive step size, with bias correction for the early steps.

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1)\, g_t, \qquad v_t = \beta_2\, v_{t-1} + (1-\beta_2)\, g_t^2$$
$$\hat{m}_t = m_t / (1-\beta_1^t), \qquad \hat{v}_t = v_t / (1-\beta_2^t), \qquad \theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

GANs use $\beta_1 = 0.5$ instead of the default 0.9. The adversarial objective is non-stationary (each network's target keeps moving), so less momentum lets the optimiser react faster and reduces oscillation. Settings: $\eta = 2 \cdot 10^{-4}$, $\beta_2 = 0.999$, with a separate optimiser for G and for D.

9.2 Learning-rate schedule: constant, then linear decay

$$\eta(e) = \eta_0 \cdot [\, 1 - \max(0,\, e + 1 - E_{const}) / (E_{decay} + 1) \,]$$

The learning rate stays at η_0 for $E_{const} = 15$ epochs, then decreases linearly towards 0 over $E_{decay} = 10$ epochs. Annealing reduces the noise of the adversarial game and lets both networks settle (`torch.optim.lr_scheduler.LambdaLR`).

9.3 Automatic Mixed Precision (AMP)

On CUDA, the forward and backward passes run in float16 inside `torch.autocast`. This roughly halves memory use and speeds up Tensor-Core matrix multiplies, which is what makes batch size 8 at 256 px fit on a 6 GB RTX 3050. Small fp16 gradients can underflow to zero, so a **GradScaler** multiplies the loss by a large factor before `backward()`, un-scales the gradients before the optimiser step, and adjusts the factor dynamically. It skips the step if it finds inf or NaN values. G and D each have their own scaler.

9.4 Checkpointing, model selection and reproducibility

- Each epoch saves `latest.pt`: G, D, both optimisers, both schedulers and the config, so training can resume exactly (`--resume`).
- `best.pt` is the checkpoint with the highest validation SSIM. This is a form of early stopping and model selection on held-out data.
- `seed_everything()` fixes the Python, NumPy and PyTorch RNGs so runs are repeatable.
- A qualitative grid (sketch | generated | real) is saved every epoch for visual monitoring.

10. GAN training stability

GAN training is a two-player game, not a single minimisation, so the losses do not have to go down steadily. The logs record D's mean probability on real images (D_real) and on fakes (D_fake) to diagnose the following failure modes.

Failure mode	Symptom	Mitigation used / available
Discriminator overpowers G	D_real → 1, D_fake → 0, G loss keeps rising, vanishing gradients	0.5 factor on L_D; non-saturating G loss; label smoothing (--real-label 0.9); LSGAN
Mode collapse	G produces the same output for many inputs	Strong conditioning on the sketch + L1 loss tie each output to its own input
Oscillation / non-convergence	Losses swing wildly, samples get worse	Adam β1 = 0.5, low LR 2e-4, BatchNorm, LR decay
Blurry outputs	Plausible layout but washed-out colours and texture	Adversarial PatchGAN loss; optional VGG perceptual loss
Checkerboard artefacts	Grid-like patterns from transposed convs	k = 4 divisible by s = 2 (even kernel overlap); more training
Over-fitting D	Validation quality drops while training D loss → 0	Small PatchGAN, data augmentation (jitter + flip)

A healthy run usually shows D_real ≈ 0.6–0.8 and D_fake ≈ 0.2–0.4, with L1 decreasing slowly. The equilibrium in theory is D = 0.5 for both.

11. Evaluation metrics

11.1 Mean Absolute Error (L1)

$$L1 = (1 / HWC)\ \Sigma\ |\ y - \hat{y}\ |, \quad \text{images scaled to } [0, 1] \quad \text{(lower is better)}$$

11.2 Peak Signal-to-Noise Ratio (PSNR)

$$MSE = (1 / HWC)\ \Sigma\ (y - \hat{y})^2, \qquad PSNR = 10 \cdot \log_{10}(MAX^2 / MSE)\ \ dB$$

A pixel-fidelity metric where higher is better. It rewards blurry 'average' predictions, which is why it is not enough on its own for generative models.

11.3 Structural Similarity (SSIM)

$$SSIM(a, b) = [(2\mu_a\mu_b + C_1)(2\sigma_{ab} + C_2)] / [(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)]$$

Compares local luminance, contrast and structure in sliding windows. Its range is [−1, 1] and higher is better. It is closer to human perception than PSNR and is used here to select the best checkpoint.

11.4 Fréchet Inception Distance (FID)

$$FID = \|\ \mu_r - \mu_g\ \|^2 + Tr(\ \Sigma_r + \Sigma_g - 2\ (\Sigma_r\ \Sigma_g)^{1/2}\ )$$

Real and generated images are passed through an ImageNet-trained Inception-v3. A Gaussian (μ, Σ) is fitted to each set's 2048-d pool features, and FID is the Fréchet (Wasserstein-2) distance between the two

Gaussians. It measures **realism and diversity** at the distribution level; lower is better. It is the standard GAN metric (Heusel et al., 2017). With only 200 validation images FID is noisy, so compare models on the same set.

11.5 Ablation study and qualitative evaluation

Train the same network with `--lambda-gan 0` (L1 only) and compare it with the full cGAN + L1. The usual result is that L1-only scores similar or higher PSNR but looks blurry and has a much worse FID, while the cGAN gives sharp, realistic texture. This isolates the contribution of adversarial training. Qualitative grids (sketch | output | ground truth) and outputs for unseen hand-drawn sketches from the web app complete the evaluation.

In our code: `image_metrics()` and `FIDCalculator` in `s2i/metrics.py`; `evaluate.py` writes `metrics.json`, and `scripts/plot_curves.py` plots the loss and metric curves.

12. Inference and deployment

- **Only G is deployed.** D exists only to train G and is thrown away afterwards.
- `model.eval()` switches BatchNorm to its running statistics and turns dropout off, so outputs are deterministic. `torch.no_grad()` skips building the autograd graph, which saves memory and time.
- The **FastAPI** server loads the checkpoint once at start-up and exposes `POST /api/generate` (PNG in, PNG out, with an X-Inference-Ms latency header) and `GET /api/health`.
- The **frontend** is a plain HTML5 canvas with pen, eraser, brush size, undo, upload and a live mode. It sends the canvas as a PNG blob through `fetch()`. All deep-learning work happens server-side in PyTorch, so the web layer is a thin client and the core of the project is the trained model.
- One forward pass of the 54 M-parameter U-Net at 256 px takes a few milliseconds on a GPU and well under a second on a CPU, which is fast enough for interactive use.

13. Hyper-parameters and concept-to-code map

Hyper-parameter	Value	Hyper-parameter	Value
Image size	256 × 256	Batch size	8
Generator	U-Net-256, 54.41 M	Discriminator	PatchGAN 70×70, 2.77 M
ngf / ndf	64 / 64	Normalisation	BatchNorm
Adversarial loss	BCE with logits	λ_{L1}	100
Optimiser	Adam (0.5, 0.999)	Learning rate	$2 \cdot 10^{-4}$
Epochs	15 constant + 10 decay	Dropout	0.5 in U1–U3
Init	N(0, 0.02)	Precision	AMP fp16 (CUDA)

Concept	File / symbol
U-Net, skip connections, dropout noise	s2i/models.py · UNetGenerator, Down, Up
PatchGAN discriminator	s2i/models.py · PatchDiscriminator
Weight initialisation N(0, 0.02)	s2i/models.py · init_weights
Adversarial loss (BCE / LSGAN), label smoothing	s2i/losses.py · GANLoss
Perceptual loss (transfer learning, VGG-16)	s2i/losses.py · VGGPerceptualLoss
XDoG / Canny edge extraction	s2i/sketch.py · xdog, canny
Paired augmentation, [-1, 1] normalisation	s2i/dataset.py · PairedSketchDataset

Concept	File / symbol
Alternating D/G updates, detach, freeze D	train.py · main loop
Adam, LambdaLR linear decay, AMP + GradScaler	train.py
PSNR, SSIM, L1, FID	s2i/metrics.py; evaluate.py
Inference, eval mode, sketch cleaning	s2i/inference.py · SketchToImage
Serving	app/server.py (FastAPI), app/static (canvas UI)

14. Viva preparation

Q. Why a GAN and not a plain CNN trained with L1/L2 loss?
Many photos are consistent with one sketch. Pixel losses are minimised by the average of those possibilities, which is a blurry image. The adversarial loss penalises any output that does not look like a real photo, which forces sharp, committed predictions.

Q. What makes it a conditional GAN?
Both G and D receive the sketch. G maps sketch → image, and D classifies the concatenated (sketch, image) pair, so it rejects realistic images that do not match the sketch.

Q. Why keep the L1 loss at all, and why λ = 100?
L1 makes the output match the ground truth in colour and layout (low frequencies) and stabilises early training. λ = 100 brings the L1 term (about 0.1) to a scale comparable with or larger than the GAN term (about 1). This is the value from the Pix2Pix paper.

Q. Why L1 instead of L2?
L2 grows quadratically and is minimised by the mean, which gives more blur. L1 is minimised by the median, is more robust to outliers and gives less blur.

Q. What does the U-Net skip connection do?
It concatenates encoder features with decoder features of the same resolution. The exact edge positions in the sketch therefore reach the output without passing through the 1×1 bottleneck. It also improves gradient flow.

Q. What is a PatchGAN and why 70×70?
A fully convolutional discriminator that outputs a 30×30 grid of real/fake logits, each with a 70×70 receptive field (derived in Section 7.2). It models local texture, has few parameters and works at any image size.

Q. Where is the random noise z?
There is no explicit z. Pix2Pix found that G learns to ignore it, so stochasticity comes from dropout in the first decoder layers.

Q. Why LeakyReLU in D but ReLU in G?
LeakyReLU keeps a gradient for negative inputs, so D can always pass useful gradients back to G. This follows the DCGAN guidelines.

Q. Why Tanh at the output and [−1, 1] normalisation?
Tanh is bounded and zero-centred. Scaling the targets to the same range makes the L1 loss consistent and keeps the data centred.

Q. Why does Adam use β1 = 0.5 here?
The GAN objective is non-stationary, so less momentum reduces oscillation and overshoot (DCGAN recommendation).

Q. Why detach the fake image in the D step?
The D loss should update only D. detach() stops gradients from flowing into G's graph, which saves compute and avoids pushing G the wrong way.

Q. What is the non-saturating loss?

Instead of minimising log(1 − D(G(x))), which has tiny gradients when D wins, G maximises log D(G(x)), i.e. BCE with label 'real'. It has the same fixed point and much stronger gradients.

Q. What is mode collapse, and is it a problem here?

It is when G maps many inputs to the same output. Strong conditioning plus the L1 loss tie each output to its own sketch, so it is much less of a problem than in unconditional GANs.

Q. How do you know the model is good?

Quantitatively: L1, PSNR and SSIM on held-out validation pairs, plus FID for realism. Qualitatively: the sample grids and outputs for new hand-drawn sketches. The L1-only ablation shows what the adversarial term adds.

Q. Why is PSNR not enough for GANs?

It is a per-pixel error and rewards blurry averages. FID compares feature distributions and correlates better with how realistic the images look to humans.

Q. What does mixed precision do?

It runs most operations in fp16 for speed and lower memory use. GradScaler scales the loss up to prevent fp16 gradient underflow, then un-scales the gradients before the optimiser step.

Q. How do hand-drawn sketches differ from training data, and how do you handle it?

Line thickness, position and scale all differ (a domain gap). The input is cropped to the strokes, centred, resized and binarised to look like the training edge maps.

Q. Limitations and future work?

One output per sketch (limited diversity), domain-specific (shoes), and 256 px resolution. Possible extensions: BicycleGAN or a style code for multi-modal outputs, pix2pixHD or SPADE for high resolution, CycleGAN for unpaired data, and latent diffusion models such as ControlNet.

Q. How is this different from CycleGAN?

Pix2Pix needs paired data and uses a supervised L1 loss. CycleGAN works with unpaired domains using two generators and a cycle-consistency loss $\|F(G(x)) - x\|_1$.

15. References

[1] I. Goodfellow et al., "Generative Adversarial Nets," NeurIPS, 2014.

[2] M. Mirza and S. Osindero, "Conditional Generative Adversarial Nets," arXiv:1411.1784, 2014.

[3] A. Radford, L. Metz and S. Chintala, "Unsupervised Representation Learning with Deep Convolutional GANs (DCGAN)," ICLR, 2016.

[4] P. Isola, J.-Y. Zhu, T. Zhou and A. A. Efros, "Image-to-Image Translation with Conditional Adversarial Networks," CVPR, 2017.

[5] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015.

[6] X. Mao et al., "Least Squares Generative Adversarial Networks," ICCV, 2017.

[7] T. Salimans et al., "Improved Techniques for Training GANs," NeurIPS, 2016.

[8] J. Johnson, A. Alahi and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," ECCV, 2016.

[9] S. Ioffe and C. Szegedy, "Batch Normalization," ICML, 2015.

[10] N. Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," JMLR, 2014.

[11] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015.

[12] P. Micikevicius et al., "Mixed Precision Training," ICLR, 2018.

[13] H. Winnemöller, J. E. Kyprianidis and S. C. Olsen, "XDoG: An eXtended Difference-of-Gaussians Compendium," Computers & Graphics, 2012.

[14] J. Canny, "A Computational Approach to Edge Detection," IEEE TPAMI, 1986.

[15] S. Xie and Z. Tu, "Holistically-Nested Edge Detection," ICCV, 2015.

[16] Z. Wang et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," IEEE TIP, 2004.

[17] M. Heusel et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," NeurIPS, 2017.

[18] A. Yu and K. Grauman, "Fine-Grained Visual Comparisons with Local Learning (UT-Zappos50K)," CVPR, 2014.